

Plan de la démo — Développer une application avec l'IA en 2026

Application de gestion de **CRA** (Compte Rendu d'Activité) — voir [README.md](#) et les user stories dans [specs/](#).
Stack : **React** (TypeScript, Vite) + **Python FastAPI** + SQLite.

Pour chaque étape, le **prompt** à donner à l'IA est indiqué dans un bloc. Les user stories de `specs/` servent de source de vérité : les prompts y font référence plutôt que de tout re-décrire. **Ordre des user stories** : elles se réalisent **dans l'ordre de leur numéro**, de US-001 à US-017. Chacune est autonome — elle ne suppose jamais une story ultérieure — et son éventuel prérequis, toujours une story précédente, est indiqué en tête de son fichier. À chaque story terminée, l'application reste démontrable en l'état.

Méthode d'implémentation : pour chaque user story, les **tests d'API sont écrits avant le code**, à partir des critères d'acceptation et du contrat figé par `architect`. On implémente ensuite jusqu'à ce qu'ils passent, puis on complète par les tests unitaires jusqu'à **70 % de couverture minimum** des deux côtés.

À partir de la phase 2, chaque user story se déroule avec les commandes **Spec Kit**, qui pilotent nos propres agents et skills : `/speckit.specify` → `/speckit.plan` (agent `architect`) → `/speckit.tasks` → `/speckit.implement` (`fastapi-dev` et `react-dev`) → `/speckit.analyze`, puis relecture par `cra-reviewer`. Le détail figure dans « Boucle standard pour chaque user story », et la constitution écrite en phase 0 est ce qui impose nos règles à Spec Kit.

Compatibilité outils : le plan fonctionne avec **Claude Code** et **GitHub Copilot**. La mémoire du projet est unique et générique (`AGENT.md`), chaque outil y étant renvoyé par un pointeur court. Les règles, skills et agents sont créés en double — même contenu, deux emplacements : `.claude/` (Claude Code) et `.github/` (Copilot). Les prompts des phases 1 à 7 sont utilisables tels quels dans les deux outils.

Phase 0 — Initialisation de l'outillage IA

Avant d'écrire la moindre ligne de code, on équipe les deux assistants pour le projet : Spec Kit, mémoire du projet, règles de codage, skills et agents spécialisés. La mémoire est un `AGENT.md` unique et générique ; les règles, skills et agents sont créés en double avec le même contenu — `.claude/` pour Claude Code, `.github/` pour GitHub Copilot. C'est ce qui garantit un code homogène pendant toute la démo, quel que soit l'outil utilisé.

Une version déjà générée de tout cet outillage est disponible dans [sample/](#) : on peut soit la rejouer avec les prompts ci-dessous, soit la copier telle quelle.

- ☐ Installer Spec Kit et initialiser le projet

Spec Kit ([github/spec-kit](#)) apporte le workflow *spec-driven development* et ses commandes `/speckit.*`, disponibles aussi bien dans Claude Code que dans GitHub Copilot. Il installe ses propres templates et sa constitution de projet, sur lesquels s'appuient les phases suivantes.

Commandes :

```
# installation de la CLI (nécessite uv)
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git

# initialisation dans le projet existant, pour les deux assistants
specify init --here --ai claude
specify init --here --ai copilot

# vérification des prérequis
specify check
```

Prompt (après installation) :

Vérifie que Spec Kit est bien installé : liste les commandes `/speckit.*` disponibles et les fichiers qu'il a créés.

Puis lance `/speckit.constitution` pour rédiger la constitution du projet à partir du `README.md` et de `PLAN.MD`. C'est la constitution qui fait obéir Spec Kit à notre outillage : elle est relue par `/speckit.plan` et `/speckit.implement`, donc tout ce qui doit s'imposer à chaque story y figure.

Elle doit poser :

- la stack React + FastAPI + SQLite et la structure backend/ frontend/ specs/
- les conventions : code et commentaires en anglais, UI et specs en français
- les règles métier dans les services backend, jamais dans un router ni un composant
- les contrôles de rôle appliqués côté serveur, jamais seulement masqués dans l'UI
- les tests d'API écrits avant le code, et 70 % de couverture minimum des deux côtés
- l'obligation d'appliquer les règles `.claude/rules/` et les skills du projet
- la répartition du travail : architect conçoit et fige le contrat d'API, fastapi-dev implémente backend/**, react-dev implémente frontend/**, cra-reviewer relit sans jamais modifier de fichier

`specify init` réécrit `CLAUDE.md` avec son propre bloc managé. Après cette étape, et après toute commande Spec Kit qui régénère le contexte agent, ouvrir `CLAUDE.md` et rétablir le pointeur vers `AGENT.md` en conservant le bloc Spec Kit à la fin.

Les 10 user stories de `specs/` sont déjà rédigées : on peut les garder telles quelles et n'utiliser Spec Kit qu'à partir de `/speckit.plan`, ou rejouer `/speckit.specify` sur une story pour montrer la génération de spécification en direct.

- ☐ Installer le serveur MCP Chrome DevTools pour la vérification de l'IHM

Les tests de composants restent en Vitest (jsdom, rapides). Le MCP Chrome DevTools sert à vérifier l'**application qui tourne** dans un vrai Chrome : le proxy Vite, l'en-tête `X-Demo-User`, les vraies charges utiles, le CSS, la console. Une suite Vitest verte ne prouve rien sur un proxy cassé.

Commandes :

```
# Claude Code
claude mcp add chrome-devtools -- npx chrome-devtools-mcp@latest

# GitHub Copilot : déclarer le même serveur dans .vscode/mcp.json
# { "servers": { "chrome-devtools": {
#     "command": "npx", "args": ["chrome-devtools-mcp@latest"] } } }
```

Prompt (après installation) :

Vérifie que le serveur MCP chrome-devtools répond : liste ses outils disponibles, ouvre une page sur <http://localhost:5173>, prends un snapshot d'accessibilité et relève les messages de console. Si les serveurs ne tournent pas encore, dis-le simplement – cette étape ne sert qu'à valider l'installation.

Un serveur MCP n'est chargé qu'au démarrage d'une session : installer celui-ci avant de commencer, puis redémarrer l'assistant, faute de quoi la vérification IHM de la phase 1 se fera sans navigateur.

L'usage détaillé (boucle snapshot → action → console → réseau, scénario de démo, format de constat) est décrit dans le skill `ui-verification` créé plus bas.

- ☐ Créer la mémoire du projet dans un `AGENT.md` générique, avec un pointeur par outil

Prompt :

Crée la mémoire du projet à partir du README.md et de PLAN.MD : contexte (démo de gestion de CRA), stack (React + Vite + TypeScript / FastAPI + SQLAlchemy + SQLite), structure du monorepo (backend/, frontend/, specs/), modèle de domaine, conventions (code en anglais, UI et specs en français), et les commandes de lancement/test des deux côtés.

Écris tout ce contenu dans un seul fichier générique AGENT.md à la racine : c'est la source unique, valable pour n'importe quel assistant. Crée ensuite deux pointeurs courts qui renvoient vers AGENT.md et ne contiennent que les emplacements propres à chaque outil (règles, skills, agents) :

- CLAUDE.md à la racine (Claude Code)
- .github/copilot-instructions.md (GitHub Copilot)

Aucune connaissance projet ne doit être dupliquée dans ces deux pointeurs.

- ☐ Créer les règles de codage, séparées en Do et Don't, pour les deux outils : `.claude/rules/{react,python}-{do,dont}.md` et `.github/instructions/{react,python}-{do,dont}.instructions.md`

Prompt :

Crée quatre règles de codage, chacune en deux exemplaires de même contenu : dans `.claude/rules/` (fichiers `.md`, pour Claude Code) et dans `.github/instructions/` (fichiers `.instructions.md` avec frontmatter `applyTo - "frontend/**"` pour react, `"backend/**"` pour python – pour GitHub Copilot) :

- react-do : les pratiques à suivre côté frontend (composants fonction, hooks, TypeScript strict, organisation pages/composants/client API, gestion d'état simple sans lib externe)
- react-dont : les anti-patterns interdits côté frontend (ex. : any en TypeScript, appels API directs dans les composants, état global inutile)
- python-do : les pratiques à suivre côté backend (FastAPI : routers fins / services / modèles SQLAlchemy, schémas Pydantic, type hints partout, tests pytest)
- python-dont : les anti-patterns interdits côté backend (ex. : logique métier dans les routers, requêtes SQL brutes, endpoints sans schéma de réponse)

Les règles doivent être courtes et actionnables, une ligne par règle.

- ☐ Créer les 10 skills (dont un facultatif), pour Claude Code et pour GitHub Copilot

Chaque skill est un mode d'emploi court et opérationnel, avec des **exemples de code réels du projet** (pas de pseudo-code) et une checklist finale. Contenu attendu skill par skill :

Skill	Contenu attendu
fastapi-endpoint	Anatomie d'un endpoint de bout en bout : router → schéma Pydantic (request/response) → service → repository. Nommage et versionnage des routes <code>/api/...</code> , codes HTTP (200/201/204/400/403/404/409), gestion d'erreurs via <code>HTTPException</code> et handler global, injection de dépendances (<code>Depends</code>) pour la session DB et l'utilisateur courant, pagination et filtres. Exemple complet sur une ressource du projet (mission).
fastapi-data-model	Modélisation SQLAlchemy du domaine CRA : entités (User, Mission, Assignment, Cra, CraEntry, Notification), relations et clés étrangères, enums (<code>CraStatus</code> , <code>EntryType</code>), contraintes d'unicité (1 CRA par consultant et par mois), migrations simples, chargement du seed au démarrage. Règles de conversion modèle ↔ schéma Pydantic.
fastapi-testing	Ordre imposé : tests d'API écrits d'abord depuis les critères d'acceptation (ils échouent tant que le code n'existe pas, c'est attendu), puis implémentation jusqu'à ce qu'ils passent — sans jamais affaiblir un test — , puis tests unitaires jusqu'au plancher de couverture. Configuration <code>--cov-fail-under=70</code> dans <code>pyproject.toml</code> , lecture de <code>term-missing</code> plutôt que du pourcentage. Tests pytest : structure <code>tests/</code> , fixtures (base SQLite en mémoire, client <code>httpx.AsyncClient</code> , utilisateurs de démo consultant/manager), tests d'API par endpoint, tests unitaires des règles métier (max 1 jour, transitions de statut, jours fériés), tests d'autorisation par rôle. Convention de nommage <code>test_<action>_<condition>_<résultat attendu></code> .
react-screen	Anatomie d'un écran : page → composants → hook d'appel API → client HTTP typé. Routing et navigation, typage strict des DTO partagés avec l'API, gestion systématique des trois états (chargement / erreur / vide), formulaires contrôlés et validation, remontée d'erreurs backend à l'utilisateur en français. Exemple complet sur un écran du projet (liste des missions).
react-design-system	Cohérence visuelle de la démo : palette et tokens (couleurs par type d'absence et par statut de CRA), typographie, espacements, composants de base réutilisables (bouton, badge de statut, carte, modale, tableau, toast), états d'interaction (hover/focus/disabled), accessibilité minimale (contrastes, labels, navigation clavier), responsive.
react-testing	Plancher de couverture 70 % configuré dans <code>vite.config.ts</code> (<code>thresholds</code>), en couvrant d'abord ce qui porte du comportement (client API, hooks, trois états). Tests Vitest + Testing Library : configuration, rendu d'un composant avec providers, requêtes par rôle accessible plutôt que par classe CSS, simulation d'interactions utilisateur, mock du client API, tests des trois états d'un écran. Un exemple de test par type (composant simple, écran avec appel API, formulaire).

Skill	Contenu attendu
ui- verification	Vérification de l'IHM avec le MCP Chrome DevTools sur l'application qui tourne : boucle <code>navigate_page</code> → <code>take_snapshot</code> (lire l'arbre d'accessibilité avant d'agir) → <code>click / fill</code> → <code>wait_for</code> → <code>list_console_messages</code> → <code>list_network_requests</code> → <code>take_screenshot</code> . Contrôles par écran (trois états réellement observés, libellés français, pas d'enum brut ni de date ISO, navigation clavier, <code>resize_page</code> à 375 px), vérification des rôles par appel API direct plutôt que par bouton masqué, scénario de démo complet, format de constat. Délimitation explicite avec Vitest : une règle testable en Vitest ou pytest y reste.
story- readiness	Contrôler une user story avant toute conception : est-elle compréhensible, unitaire, testable, complète, bornée et implémentable ? Délègue à l'agent <code>business-analyst</code> et produit un rapport par story dans <code>docs/revues/</code> , depuis le template <code>template/revue.md</code> . Verdict en trois états — Prête / À clarifier / Non prête — qui décide si l'on enchaîne sur <code>architect</code> ou si l'on retourne poser des questions. Ne modifie jamais <code>specs/</code> : une reformulation est une proposition, pas une correction appliquée.
architecture- pdf	Produire <code>docs/architecture.pdf</code> à partir d'un template livré avec le skill (<code>template/architecture.md</code> et <code>template/print.css</code>) : document d'architecture assemblé depuis le dépôt — <code>AGENT.md</code> , les notes de <code>docs/architecture/</code> , les ADR, et ce que le code contient réellement (routers, modèles, écrans, chiffres de tests). Plan en 9 sections, du contexte aux stories restant à construire, avec mention explicite de ce qui est conçu mais pas encore construit. Rendu par Chrome headless (<code>--print-to-pdf</code>) après conversion Markdown → HTML avec un moteur CommonMark ; WeasyPrint échoue sur macOS faute de <code>libgobject</code> , et <code>python-markdown</code> casse les blocs de code imbriqués. Neuf sections figées par le template — aucune ajoutée, aucune retirée — et vérification obligatoire du rendu par capture avant livraison.
implement-us (facultatif)	Workflow d'implémentation d'une user story de bout en bout, en 7 étapes : (0) cadrage et extraction des critères d'acceptation, (1) conception par <code>architect</code> — le contrat d'API est le point de passage obligé, (2) tests d'API écrits d'abord , un par critère d'acceptation, (3) implémentation parallèle par <code>fastapi-dev</code> et <code>react-dev</code> jusqu'à ce qu'ils passent, sans jamais affaiblir un test, puis tests unitaires jusqu'au plancher, (4) vérification <code>pytest --cov-fail-under=70</code> , <code>ruff</code> , <code>npm test --coverage</code> , <code>build</code> , <code>lint</code> , plus la vérification navigateur, (5) relecture par <code>cra-reviewer</code> avec boucle limitée à 2 tours, (6) case cochée dans <code>PLAN.MD</code> et commit. Interchangeable : Spec Kit (<code>/speckit.specify</code> → <code>.plan</code> → <code>.tasks</code> → <code>.implement</code>) ou du prompting manuel remplissent le même rôle ; ce qui compte est l'invariant « conception d'abord, un agent par côté, relecture avant de clore ».

Prompt :

Crée les 10 skills du projet, en te basant sur le tableau des skills de PLAN.MD (phase 0) pour le contenu attendu de chacun : fastapi-endpoint, fastapi-data-model, fastapi-testing, react-screen, react-design-system, react-testing, ui-verification, architecture-pdf, story-readiness et implement-us.

Règles communes à tous les skills :

- court et opérationnel (100 à 160 lignes selon le sujet), pas de blabla théorique : une fiche de référence tient en ~100 lignes, un skill de test ou de workflow en demande davantage – c'est la densité qui compte, pas le compteur
- des exemples de code réels tirés du domaine CRA du projet, pas du pseudo-code
- une checklist finale "avant de considérer la tâche terminée"
- renvoyer vers les règles Do/Don't correspondantes plutôt que les recopier
- contenu rédigé en anglais, comme le code

Deux skills livrent en plus un sous-dossier template/ à côté de leur SKILL.md, que le skill remplit au lieu de repartir d'une page blanche :

- architecture-pdf : template/architecture.md (les 9 sections et leurs {{PLACEHOLDER}}) et template/print.css (la feuille de style d'impression, partagée par les documents du projet)
- story-readiness : template/revue.md, template/print.css et template/render.py (Markdown vers HTML avec un moteur CommonMark)

Chaque skill existe en deux exemplaires de même contenu, au même format SKILL.md avec frontmatter name + description (la description doit dire quand déclencher le skill) :

- .claude/skills/<nom>/SKILL.md (Claude Code)
- .github/skills/<nom>/SKILL.md (GitHub Copilot)

Les sous-dossiers template/ sont copiés à l'identique des deux côtés.

- ☐ Créer les 5 agents spécialisés (business-analyst, architect, fastapi-dev, react-dev, cra-reviewer) pour les deux outils

Chaque agent a un périmètre de fichiers, des règles et skills à charger, un mode opératoire imposé et une définition de « terminé ». Enchaînement type sur une user story : architect → fastapi-dev et react-dev en parallèle sur le contrat défini → cra-reviewer . Contenu attendu agent par agent :

Agent	Contenu attendu
business-analyst	Rôle : analyste métier, seul à contester le besoin plutôt que la solution. Périmètre : lecture seule sur specs/ , écrit uniquement dans docs/revues/ . Modèle : opus . Mode opératoire : juger six axes — compréhensible, unitaire, testable, complète, bornée, implémentable — en cherchant en priorité les manques coûteux : règle métier sans son cas d'échec, rôle sans son refus, libellé français absent, valeur limite non tranchée, donnée supposée exister, contradiction avec une autre story. Sortie : un rapport par story, chaque constat citant le critère mot pour mot avec une question fermée et une reformulation proposée. Terminé quand : les six axes sont justifiés, le verdict est cohérent avec les constats, et specs/ est intact.
architect	Rôle : architecte logiciel, conçoit avant que quiconque code. Périmètre : écrit uniquement dans docs/architecture/ et docs/adr/ , jamais de code de production ni de modification des specs/ . Charge : la user story, AGENT.md , les ADR existantes, le code en place. Mode opératoire : (1) cadrer le problème et lister les questions ouvertes, (2) inventorier l'existant avant d'ajouter, (3) concevoir le modèle de domaine et le lieu d'application de chaque invariant, (4) figer le contrat d'API (route, payloads, codes, erreurs, rôle) qui débloque le frontend avant le backend, (5) définir la structure frontend, (6) donner au moins deux options avec arbitrage explicite

Agent	Contenu attendu
	sur chaque choix structurant, (7) écrire la note de conception et une ADR par décision, (8) répartir le travail entre <code>fastapi-dev</code> et <code>react-dev</code> . Principes : simplicité adaptée à une démo (pas de CQRS ni de microservices), règles métier côté backend, contract first, valeurs dérivées jamais persistées, choix réversibles. Terminé quand : chaque critère d'acceptation est traçable vers un endpoint ou un écran, le contrat d'API est complet, chaque décision a son ADR, rien n'est écrit hors de <code>docs/</code> .
<code>fastapi-dev</code>	Rôle : développeur backend Python/FastAPI. Périmètre : <code>backend/**</code> uniquement (ne touche jamais à <code>frontend/**</code>). Charge : <code>python-do</code> , <code>python-dont</code> , skills <code>fastapi-endpoint</code> , <code>fastapi-data-model</code> , <code>fastapi-testing</code> . Mode opératoire test-first : (1) lire la user story, (2) identifier les endpoints et le modèle concernés, (3) écrire les tests d'API d'abord (un par critère, nommé d'après lui), (4) modèle, (5) schémas, (6) service, (7) router jusqu'à ce qu'ils passent — un test faux signale un contrat faux, on remonte à <code>architect</code> plutôt que d'éditer l'assertion, (8) tests unitaires jusqu'à 70 % de couverture. Terminé quand : chaque critère est tracé vers un test d'API nommé, écrit avant le code, les tests passent, la couverture atteint 70 %, les contrôles de rôle sont faits côté backend, l'OpenAPI est cohérent.
<code>react-dev</code>	Rôle : développeur frontend React/TypeScript. Périmètre : <code>frontend/**</code> uniquement. Charge : <code>react-do</code> , <code>react-dont</code> , skills <code>react-screen</code> , <code>react-design-system</code> , <code>react-testing</code> . Mode opératoire : (1) lire la user story dans <code>specs/</code> , (2) vérifier les endpoints disponibles dans l'OpenAPI du backend, (3) typer les DTO, (4) écrire le hook d'appel API puis les composants puis la page, (5) traiter les trois états (chargement/erreur/vide), (6) écrire les tests. Terminé quand : l'écran est conforme à la US, les libellés sont en français, aucun <code>any</code> , les tests passent avec 70 % de couverture, et l'écran a été vu tourner dans Chrome.
<code>cra-reviewer</code>	Rôle : relecteur qualité, en lecture seule (ne modifie aucun fichier, produit un rapport). Vérifie aussi l'IHM dans un vrai navigateur via le MCP Chrome DevTools et le skill <code>ui-verification</code> — piloter le navigateur reste de l'observation : il constate, il ne corrige jamais ; si un serveur ne démarre pas, c'est un constat, pas une omission silencieuse. Périmètre : tout le repo. Charge : les 4 fichiers de règles Do/Don't et la user story concernée. Mode opératoire : comparer le code produit aux critères d'acceptation de la US, puis aux règles Do/Don't, puis chercher les oublis classiques (gestion d'erreur absente, contrôle de rôle uniquement côté UI, règle métier non testée, libellés en anglais dans l'UI). Sortie : liste de constats classés bloquant / à corriger / suggestion, avec fichier et ligne.

Prompt :

Crée les 5 agents du projet, en te basant sur le tableau des agents de PLAN.MD (phase 0) pour le contenu attendu de chacun : business-analyst, architect, fastapi-dev, react-dev, cra-reviewer.

Chaque fichier d'agent doit contenir explicitement :

- le modèle à utiliser, dans le frontmatter : `opus` pour business-analyst, architect et cra-reviewer (conception et relecture, où le raisonnement paie), `sonnet` pour fastapi-dev et react-dev (implémentation guidée par un contrat et des règles). Côté Copilot, mettre l'identifiant complet : `claude-opus-5` / `claude-sonnet-5`.
- son rôle et son périmètre de fichiers (ce qu'il ne doit jamais toucher)
- les règles et skills qu'il charge avant de commencer
- son mode opératoire numéroté, étape par étape
- sa définition de "terminé" sous forme de checklist vérifiable
- la consigne de toujours lire la user story dans specs/ avant de coder
- le rappel des conventions : code et commentaires en anglais, UI en français

Chaque agent existe en deux exemplaires de même contenu :

- .claude/agents/<nom>.md avec frontmatter name + description + model + tools (Claude Code) ; business-analyst n'a que Read, Grep et Glob et n'écrit que dans docs/revues/, cra-reviewer n'a que des outils de lecture plus ceux du MCP chrome-devtools, architect n'écrit que dans docs/
- .github/agents/<nom>.agent.md (GitHub Copilot)

- ☐ Vérifier l'outillage des deux côtés : permissions Claude Code, chargement des instructions Copilot

Prompt :

Configure .claude/settings.json avec les permissions utiles au projet (npm, uvicorn, pytest) pour limiter les demandes d'autorisation.

Valide ensuite le frontmatter YAML de CHAQUE skill et agent créé (un « : » non échappé dans une description suffit à le rendre invalide, et le fichier ne se charge alors pas). Vérifie aussi que chaque agent porte bien un `model`, et que chaque chemin cité dans un skill ou un agent existe réellement.

Puis vérifie l'outillage complet :

AGENT.md et ses deux pointeurs, les 4 règles Do/Don't, les 10 skills et les 5 agents existent bien (skills et agents en double) (.claude/ et .github/) avec un contenu identique, les frontmatter sont valides, et aucun skill ou agent ne référence un fichier inexistant. Vérifie aussi que les fichiers .github/ (copilot-instructions.md, instructions/, skills/, agents/) sont conformes aux conventions GitHub Copilot. Termine par un récapitulatif sous forme de tableau : élément, chemin Claude Code, chemin Copilot, statut.

Phase 1 — Socle du projet

- ☐ Initialiser le monorepo : `backend/` (FastAPI, SQLAlchemy, SQLite) et `frontend/` (Vite + React + TypeScript)

Prompt :

```
Lis le README.md et les user stories dans specs/. Fais d'abord concevoir le socle par
l'agent architect (arborescences, contrat de l'endpoint /api/health, conventions
partagées), puis implémente avec fastapi-dev et react-dev en parallèle.

Initialise le squelette : un dossier backend/ (FastAPI + SQLAlchemy + SQLite, structure
routers/services/modèles) et un dossier frontend/ (Vite + React + TypeScript, structure
pages/composants/client API). Aucune fonctionnalité métier : la structure, la config et
un écran qui affiche l'état de l'API, avec ses trois états.

Ce socle doit poser dès maintenant les choix qu'on ne peut plus rattraper ensuite :
- une base CamelModel dans schemas/common.py : snake_case en Python, camelCase sur le fil
- le contrat d'erreur {"detail": "<phrase française>"} sur TOUTES les réponses non-2xx,
  y compris les 500 (handler sur Exception) et les 422 (aplatir la liste de Pydantic)
- le plancher de couverture outillé : pytest-cov avec --cov-fail-under=70 côté backend,
  @vittest/coverage-v8 avec thresholds à 70 côté frontend
- l'interdiction de any réellement appliquée par le linter (oxlint ne l'active pas par
  défaut) : typescript/no-explicit-any et ban-ts-comment en error
- le proxy Vite /api vers localhost:8000, et tout se teste via le port 5173

Termine par le point de synchro : les deux serveurs démarrés, la page ouverte sur
localhost:5173, l'état de l'API affiché en français.
```

- ☐ Jeu de données seed : 2 consultants + 1 manager (équipe du manager), missions d'exemple

Prompt :

```
Crée le jeu de données seed du backend : 2 consultants (Jean Dupont, Marie Martin)
et 1 manager (Paul Durand) qui a les 2 consultants dans son équipe, plus 3 missions
d'exemple affectées aux consultants. Le seed est chargé automatiquement au démarrage
si la base est vide.
```

Boucle standard pour chaque user story

Toutes les stories des phases 2 à 6 se déroulent de la même façon : les commandes Spec Kit portent le cycle, nos agents et nos skills font le travail. La constitution écrite en phase 0 est ce qui lie les deux — Spec Kit la relit à chaque `/speckit.plan` et `/speckit.implement`.

Étape	Commande	Qui travaille
Revue de la demande	skill <code>story-readiness</code>	agent <code>business-analyst</code> : la story est-elle implémentable telle quelle ? Verdict Non prête = on ne conçoit pas, on retourne poser les questions
Spécification	<code>/speckit.specify</code>	reprend la user story de <code>specs/</code> telle quelle ; ne pas la réécrire
Clarification	<code>/speckit.clarify</code> (si <i>besoin</i>)	lève les zones d'ombre avant de concevoir
Conception	<code>/speckit.plan</code>	agent <code>architect</code> : modèle, contrat d'API, ADR, découpage du travail

Étape	Commande	Qui travaille
Découpage	<code>/speckit.tasks</code>	tâches ordonnées, les tests d'API en premier
Implémentation	<code>/speckit.implement</code>	<code>fastapi-dev</code> sur <code>backend/**</code> , <code>react-dev</code> sur <code>frontend/**</code>
Cohérence	<code>/speckit.analyze</code>	vérifie spec ↔ plan ↔ tasks avant de clore
Relecture	agent <code>cra-reviewer</code>	constats classés, vérification dans le navigateur

Chaque story ci-dessous donne **les deux** : un **prompt** qui énonce l'intention et le point d'attention propre à la story, et les **commandes Spec Kit** qui déroulent le cycle. Le prompt dit quoi obtenir, les commandes disent comment y aller — on les utilise ensemble, le prompt servant de cadrage avant `/speckit.specify` et de rappel pendant la relecture.

Le squelette des commandes est toujours le même — seul le fichier de story change :

Utilise le skill `story-readiness` sur `specs/US-0XX-....md` et donne-moi le verdict avant d'aller plus loin.

`/speckit.specify` Reprends la user story `specs/US-0XX-....md` telle quelle, sans en élargir le périmètre ni anticiper une story ultérieure.

`/speckit.plan` Utilise l'agent `architect`. Applique la constitution du projet, les règles `.claude/rules/` et les skills `fastapi-endpoint`, `fastapi-data-model` et `react-screen`. Fige le contrat d'API avant tout code.

`/speckit.tasks` Ordonne les tâches en plaçant les tests d'API avant l'implémentation, un test par critère d'acceptation.

`/speckit.implement` Confie le backend à `fastapi-dev` et le frontend à `react-dev`, chacun dans son périmètre. Termine par `pytest` avec le seuil de couverture, `npm test`, `build` et `lint`.

`/speckit.analyze`

Puis relis avec l'agent `cra-reviewer` et corrige les constats bloquants.

Phase 2 — Identité

- **US-001** — Se connecter en choisissant un profil de démo

Prompt :

Revois d'abord la story avec le skill `story-readiness`, puis implémente `specs/US-001-connexion-profil.md` en suivant le processus du projet : conception par `architect`, tests d'API écrits d'abord, puis implémentation par `fastapi-dev` et `react-dev`, puis relecture par `cra-reviewer`.

Commandes Spec Kit :

Skill story-readiness sur specs/US-001-connexion-profil.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-001-connexion-profil.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-002** — Voir qui je suis et me déconnecter

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-002-identite-deconnexion.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.

Commandes Spec Kit :

Skill story-readiness sur specs/US-002-identite-deconnexion.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-002-identite-deconnexion.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

Phase 3 — Missions

- ☐ **US-003** — Créer une mission

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-003-creer-mission.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.
Le contrôle du rôle Manager doit être vérifié par un test d'API, pas seulement masqué dans l'UI.

Commandes Spec Kit :

Skill story-readiness sur specs/US-003-creer-mission.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-003-creer-mission.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-004** — Consulter et filtrer la liste des missions

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-004-lister-missions.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.
N'oublie pas l'état vide : un tableau sans ligne et sans message se lit comme un bug en démonstration.

Commandes Spec Kit :

Skill story-readiness sur specs/US-004-lister-missions.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-004-lister-missions.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-005** — Affecter des consultants à une mission

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-005-affecter-consultants.md en suivant le processus du projet : conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.

Commandes Spec Kit :

Skill story-readiness sur specs/US-005-affecter-consultants.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-005-affecter-consultants.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-006** — Clôturer une mission

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-006-cloturer-mission.md en suivant le processus du projet : conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.

Commandes Spec Kit :

Skill story-readiness sur specs/US-006-cloturer-mission.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-006-cloturer-mission.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

Phase 4 — Saisie du CRA

- ☐ **US-007** — Afficher le calendrier du mois

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-007-calendrier-mensuel.md en suivant le processus du projet : conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.
Soigne cet écran : c'est celui que la démo montre le plus. Teste le calcul des jours fériés, fériés mobiles compris.

Commandes Spec Kit :

Skill story-readiness sur specs/US-007-calendrier-mensuel.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-007-calendrier-mensuel.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-008** — Saisir une journée travaillée sur une mission

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-008-saisir-journee.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.

Commandes Spec Kit :

Skill story-readiness sur specs/US-008-saisir-journee.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-008-saisir-journee.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-009** — Répartir une journée en demi-journées

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-009-demi-journees.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.
La règle « somme d'un jour ≤ 1 » se teste sur le service, sans passer par HTTP.

Commandes Spec Kit :

Skill story-readiness sur specs/US-009-demi-journees.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-009-demi-journees.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-010** — Déclarer une absence

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-010-declarer-absence.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.

Commandes Spec Kit :

Skill story-readiness sur specs/US-010-declarer-absence.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-010-declarer-absence.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-011** — Voir le récapitulatif du mois

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-011-recapitulatif-mensuel.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.
Les totaux se calculent à la lecture ; ne les stocke pas.

Commandes Spec Kit :

Skill story-readiness sur specs/US-011-recapitulatif-mensuel.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-011-recapitulatif-mensuel.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

Phase 5 — Soumission et validation

- ☐ **US-012** — Soumettre son CRA

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-012-soumettre-cra.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.
Teste que toute modification est refusée par le serveur une fois le CRA soumis.

Commandes Spec Kit :

Skill story-readiness sur specs/US-012-soumettre-cra.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-012-soumettre-cra.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-013** — Annuler sa soumission

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-013-annuler-soumission.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.

Commandes Spec Kit :

Skill story-readiness sur specs/US-013-annuler-soumission.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-013-annuler-soumission.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-014** — Consulter les CRA en attente de validation

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-014-lister-cra-en-attente.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.
Vérifie par un appel direct qu'un manager ne voit pas les CRA d'une autre équipe.

Commandes Spec Kit :

Skill story-readiness sur specs/US-014-lister-cra-en-attente.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-014-lister-cra-en-attente.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-015** — Valider un CRA

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-015-valider-cra.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.

Commandes Spec Kit :

Skill story-readiness sur specs/US-015-valider-cra.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-015-valider-cra.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

- ☐ **US-016** — Refuser un CRA avec un commentaire

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-016-refuser-cra.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.

Commandes Spec Kit :

Skill story-readiness sur specs/US-016-refuser-cra.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-016-refuser-cra.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

Phase 6 — Tableau de bord

- ☐ **US-017** — Tableau de bord du consultant

Prompt :

Revois d'abord la story avec le skill story-readiness, puis implémente specs/US-017-tableau-de-bord-consultant.md en suivant le processus du projet :
conception par architect, tests d'API écrits d'abord, puis implémentation par fastapi-dev et react-dev, puis relecture par cra-reviewer.

Commandes Spec Kit :

Skill story-readiness sur specs/US-017-tableau-de-bord-consultant.md – vérifier le verdict avant de continuer.

/speckit.specify Reprends specs/US-017-tableau-de-bord-consultant.md telle quelle, sans élargir son périmètre ni anticiper une story ultérieure.

/speckit.plan Utilise l'agent architect, la constitution du projet, les règles .claude/rules/ et les skills du projet. Fige le contrat d'API avant tout code.

/speckit.tasks Place les tests d'API avant l'implémentation, un test par critère d'acceptation.

/speckit.implement Backend par fastapi-dev, frontend par react-dev, chacun dans son périmètre.

/speckit.analyze

Puis relis avec l'agent cra-reviewer et corrige les constats bloquants.

Phase 7 – Démonstration

- Scénario de démo bout en bout : Jean saisit et soumet → Paul refuse → Jean corrige et resoumet → Paul valide

Prompt :

Lance le backend et le frontend, puis déroule ce scénario dans le navigateur et corrige tout ce qui bloque : Jean se connecte, saisit son mois (une journée sur une mission, une journée partagée en deux demi-journées, une absence), vérifie son récapitulatif et soumet ; Paul se connecte, voit le CRA en attente, le refuse avec un commentaire ; Jean voit le motif, corrige et resoumet ; Paul valide ; Jean constate son CRA en « Validé » sur son tableau de bord.

- Vérifier les refus d'accès : un consultant ne peut pas emprunter les actions du manager

Prompt :

Vérifie les contrôles de rôle par des appels directs à l'API, pas seulement par l'absence de boutons : en tant que Jean (consultant), la création de mission, la liste des CRA en attente, la validation et le refus doivent tous répondre 403. En tant que Paul, un CRA d'un consultant hors de son équipe doit répondre 404.

- Script/notes de présentation : quelles parties ont été générées par l'IA et comment (specs → code → tests)

Prompt :

Rédige un script de présentation de 10 minutes pour cette démo : le déroulé specs → plan → prompts → code → tests, avec pour chaque phase ce que l'IA a généré et les points à montrer à l'écran.

- ☐ Nettoyage final : README d'installation (`make dev` ou équivalent), données seed réalistes

Prompt :

Finalise le projet : complète le README avec les instructions d'installation et de lancement (une commande unique type `make dev` si possible), vérifie que les données seed sont réalistes, et fais un dernier passage de nettoyage du code.